

ARTICLE

Optimized Neural Fusion Architecture for Secure Data Validation and Cryptographic Key Regulation in Advanced Cloud Ecosystems

Niranchana Radhakrishnan^{1,*}, C. Viji² and Balusamy Nachiappan³

¹Assistant Professor, Department of Computer Science and Engineering, Alliance School of Advanced Computing, Alliance University, Anekal, Bengaluru, Karnataka 562106, India.

²Associate Professor, Alliance School of Advanced Computing, Alliance University, Anekal, Bengaluru, Karnataka 562106, India.

³Tech Lead, Prologis, USA.

*Corresponding Author: Niranchana Radhakrishnan. Email: niranchana.r14@gmail.com

Received: 07 July 2024; Accepted: 15 September 2024; Published: 30 December 2024

ABSTRACT: Cloud computing allows Internet users to access highly scalable services based on their needs. Cloud services feature a major advantage, which allows users' data to be processed remotely in machines that users have little knowledge about, and these are neither owned nor operated by them. This innovative technology is undoubtedly advantageous; however, there is an element of fear where users have no control over their own information. In addition, this loss of control can allow attackers to hack user information and thus become a major obstacle while embracing cloud services. To resolve this issue first, a new Centralized Cloud Information Accountability Integrity with Imperialist Competitive Key Generation Algorithm (CCIAI-ICKGA) and attack detection has been suggested to check and track actual usage of users' data that is part of the cloud environment. Second, every end user maintains a set of both public information-related and private information-related keys dependent upon Ciphertext-Policy Attribute-Based Encryption (CP-ABE) with key generation while deploying the ICKGA and trapdoor generator. Third, at the user and cloud levels, using a trapdoor generator, data integrity is maintained. Fourth, the Dynamically Weighted Ensemble Neural Networks (DWENN) classifier is deployed to enable effective attack detection. Log records structure is updated as per the above to additionally guarantee user data integrity as well as dependability. Framework extension is carried out to execute a security analysis that helps in the detection of all possible attacks. Inferences are finally drawn on the basis of novel experiments, and a comprehensive and extensive evaluation of the system's performance is done to ensure parameters set are met. Results additionally provide the basis to carry out further experimental research, and they also show the competence and efficacy of the CCIAI-ICKGA framework proposed in this research.

KEYWORDS: Cloud Computing (CC), Accountability, Data sharing, Trapdoor Generator, Centralized Cloud Information Accountability (CCIA), Imperialist Competitive Key Generation Algorithm (ICKGA), Attack Detection, Dynamically Weighted Ensemble Neural Networks (DWENN), Extensible Markup Language (XML), Java Archive (JAR).

1 Introduction

Cloud computing offers an innovative approach that supplements the present consumption and delivery model for Information Technology (IT) services that are dependent on the Internet. This is done by offering dynamically scalable services and generally virtualized resources over the Internet.



As of the present time, there exists a host of well-known marketable and distinct cloud computing services, which include Amazon, Google, Microsoft, Yahoo, and Salesforce [1]. Users determine the details of the services to be provided, as it is not mandatory for them to possess expertise regarding the technological infrastructure. This is because users do not actually know the workings of machines that process and host their data. Though users benefit from the new cloud computing services and technology, their concern about having no control over their own information cannot be negated. Over the clouds, the data processing is usually outsourced, resulting in many problems associated with reliability and managing personally identifiable information. These inhibitions are a major concern that users face while adopting cloud services [2].

Data calculations and loss of control over data engender security concerns for companies, which disrupts the ease of access and extensive adaptability and technological abilities of a public cloud. Cloud computing users face loss of control over information while using the storage platform to overcome their fears, maintain and access control over data (distinct data and the data shared among a set of users via the open cloud) [3]. Additionally, both data privacy and data confidentiality are highly recommended to customers [4]. Management of data confidentiality by users also prevents the respective cloud to learn any information and gathering relevant customer-related data.

Cryptography is a distinctive tool that ensures data confidentiality and user privacy services [4]. Data Encryption is usually performed before cloud storage. Access control, key management, encryption, and decryption processes responsibility lies with customers for guaranteeing data security [5]. Though if data is required to be shared in groups, cryptographic services must be flexible so that many users can be managed simultaneously. Keys and Access control must be managed to ensure and safeguard data privacy and confidentiality [6]. In a group, data handling is comparatively advantageous than a two-party communication or handling of data that belongs to an individual user. Present, leaving, and freshly joined group members are likely to pose an internal danger that can possibly violate data privacy and data confidentiality [6]. Generally, insider threats pose more collateral damage to data and information as they are initiated by trusted entities that have access. Usually, existing members trust insider entities, and research communities attempt to resolve issues posed by outsider attackers. Hence, several security issues surface on account of various users in any group. In order to address users' concerns, an effective attack detection mechanism must be developed and deployed so that end users who access the cloud can easily evaluate their respective data usage.

Attackers can possibly easily infiltrate by influencing users and getting them to trust unreliable cloud services by using multiple accounts, generating false and multiple transactions (e.g., creation of several virtual machines for a shorter time duration), and providing misleading trust feedback. This form of manipulation creates barriers and does not allow users to recognize reliable providers.

Attack detection includes host of challenges that include i) Consumers Dynamism wherein new users attach to the cloud environment and old users detach from the environment in an on-going basis making feedback collusion detection challenging, ii) Multiplicity of Identities refers to users with numerous accounts for a specific cloud service that does not allow Sybil attack detection and cannot be performed on account of several identities deployed to generate wrong information [7], iii) Attackers Behaviors prediction of malicious behaviors is tough as it can take occur during a longer or shorter time duration (i.e., strategic vs. occasional behaviors) [8], and iv) Consumers' Privacy is where detection of attack can cause privacy breaches in udders information especially when sensitive information is involved.

Many methods were used to mitigate high-rate attacks, like establishing a speed floor of network flow, request counting, bandwidth limitation, etc. As these attacks deploy only a bare minimum of resources, their adoption of Cloud Computing becomes a major challenge [9]. Cloud environment entails a large number of attacks that can happen at the same time and target damaging services [10].

The research proposes an innovative, highly Centralized Cloud Information Accountability Integrity with Imperialist Competitive Key Generation Algorithm (CCIAI-ICKGA) and attack detection that keeps a tab on the cloud environment and the actual end user data usage therein. Every user maintains a set of both public and private keys dependent upon Ciphertext-Policy Attribute-Based Encryption (CP-ABE) with key generation while deploying the ICKGA and trapdoor generator. Thirdly, using the trapdoor generator, data integrity is maintained at the user level and cloud server level. Fourthly, the Dynamically Weighted Ensemble Neural Networks (DWENN) classifier is used to carry out attack detection. Log records structure is updated as per the above to additionally guarantee user data integrity as well as dependability. Framework extension is carried out to execute a security analysis that helps in the detection of all possible attacks. Inferences are finally drawn based on novel experiments, and a comprehensive and extensive assessment of the system's performance is done to ensure parameters set are met.

2 Literature Review

Ze Cheng He et al [11] suggested a Distributed Denial of Service (DDoS) detection system based on the basis of machine learning methods. The system is configured to allow easy implementation on the Cloud provider's side to facilitate early detection of DDoS attacks that have been acquired from virtual machines of the Cloud. The system provides leverage on the statistical information obtained from both the hypervisor and the virtual machines of the Cloud server, so that network packages may be prevented from being moved out to the external network. Analysis and Evaluation of nine machine learning algorithms is done, and on the basis of the same, the most apt is selected based on the detection performances. The suggested approach does not degrade performance, and its extension to broader DOS attacks is possible.

Sreeram et al [12] in their study have suggested a Bio-Inspired Anomaly-based Application Layer DDoS attack (App-DDoS attack) detection to attain quick and early detection. The system is one that is a bio-influenced bat algorithm that detects the Hypertext Transfer Protocol (HTTP) DDoS attacks. System evaluation was carried out using the dataset from the Center for Applied Internet Data Analysis (CAIDA). Satisfactory results were attained by the system to deliver results HTTP flooding attacks detection.

Irfan et al [13] in their study have suggested a detection system of HTTP DDoS flooding attacks that is drawn on the basis of machine learning techniques. Four machine learning classifiers, namely, Naive Bayes (NBs), Multilayer Perceptron (MLP), Support Vector Machine (SVM), and Decision Trees (DTs), are evaluated using a local dataset. Satisfactory results were attained by the system regarding network traffic data that had been collected.

In their study, Choi et al [14] have presented a DDoS attack detection employing HTTP packet pattern and rule engine in a Cloud Computing environment. This technique integrates HTTP GET flooding amongst DDoS attacks and Map Reduce processing, enabling quick attack detection to be achieved in a CC scenario. It guarantees system availability to ensure precise and correct detection of HTTP Guaranteed Entrance Token (GET) flooding. In comparison with the Snort Intrusion Detection System (IDS), this technique takes into account processing time and is more reliable, specifically in the case where congestion increases in the Cloud infrastructure.

Aiello et al [15] resolved the issue of detection of "Slow" Denial of Service attacks with their approach. The issue is quite challenging in relation to the decreasing level of bandwidth that is generated by attacks. Hence, an innovative detection technique analyzes the key spectral characteristics of traffic over small time horizons. The inspection of the packet is not mandatory. Extrapolated data relates to actual traffic traces, extended over the Local Area Network of the Institute. Several threats have been analyzed and taken into consideration. Inferences clearly show how the methodology adopted is far more reliable and contextually relevant.

Noor et al [16], to address the detection of reputation attacks, have suggested a methodology that allows consumers to make an effective identification of reliable cloud services. A credibility model has been introduced, which detects the misleading trust feedback acquired from collusion attacks and identifies Sybil attacks as well, either within a longer time duration of time or a shorter one. A massive set of the user's trust feedback has been collated, which is based on practical cloud services and helps to assess and exhibit the efficiency and suitability of the technique suggested, and highlights its potential for identifying such dangerous behaviors.

Sundareswaran et al [17] suggest a hugely decentralized information accountability framework deployed in the cloud that helps in tracking users' data actual usage. Specifically, the object-centered approach suggested facilitates enclosing the logging technique along with the data and policies of the users. JAR programmable capabilities are balanced to engender a dynamic and traveling object, and this also ensures that any attempt made to access the data of the users' triggers authentication and automatic logging that is localized to the JARs. The control of the User is further fortified, and it additionally offers distributed auditing mechanisms. Results additionally provide the basis to carry out further experimental research, and they also show the competence and efficacy of the approach deployed.

Garkoti et al [18] offered a detective approach in the case of healthcare organizations. The main aim in this approach is the detection of insider attacks so as to eliminate and prevent fake examinations done on the health records of the patient, and simultaneously ensure data accountability of data utilization. Watermarking helps in the detection of changes in the internal attack; however, it does not offer data usage accountability. The approach integrates the functions of cryptographic methods and watermarking, and an accountability framework to ensure transparency with regard to the usage of the patient's data.

Ko et al [19] in their study have presented the analysis of the issues and challenges that are faced while attaining a trusted cloud by deploying investigative controls, and presenting the Trust Cloud framework, which also ensures accountability in cloud computing is attained using technical and policy-based techniques. Hwang and Li [20] have mentioned the trust-overlay network over different data centers for the implementation of the reputation system for the establishment of trust between service providers and data owners. Shared data objects and widely distributed software modules are protected by deploying data coloring and software watermarking. Data coloring and software marking approaches protect multi-way authentications, facilitate a single sign-on in the cloud, and strengthen access control for critical information in both public and private-owned clouds.

3 Proposed Methodology

In this study, an overview of Centralized Cloud Information Accountability Integrity with Imperialist Competitive Key Generation Algorithm (CCIAI-ICKGA) and attack detection that meets the security requirements of the cloud data owner is provided. Dynamically Weighted Ensemble Neural Networks (DWENN) classifier is used to execute attack detection. JAR creation is executed first on the data of every cloud user. Post JAR creation files conversion is executed by utilizing data conversion. A JAR file consists of a group of regular access control policies that clearly identify if and how the cloud servers and probably the stakeholders who are essentially the end users or corporates accessing cloud computing services are authenticated for accessing the data present in the cloud. Every user maintains a set of both public and private keys dependent upon CP-ABE with key generation [17], deploying ICKGA. Data integrity in the cloud server and user level is performed trapdoor generator. Encrypted log files are then decrypted, and their data integrity is also confirmed through the trapdoor generator. Simultaneously, attack detection is carried out during this phase of decryption. It may be deduced that an efficient methodology has been adopted for enhancing data accountability and data integrity through the utilization of JAR files, almost for the very first time.

These are accessible by certified stakeholders or those who own the data to access the cloud as and when required to facilitate auditing by deploying the log harmonizer (step 5 in Figure 1). Furthermore, particular error correction information is forwarded to the log harmonizer to address issues related to probable log file corruption.

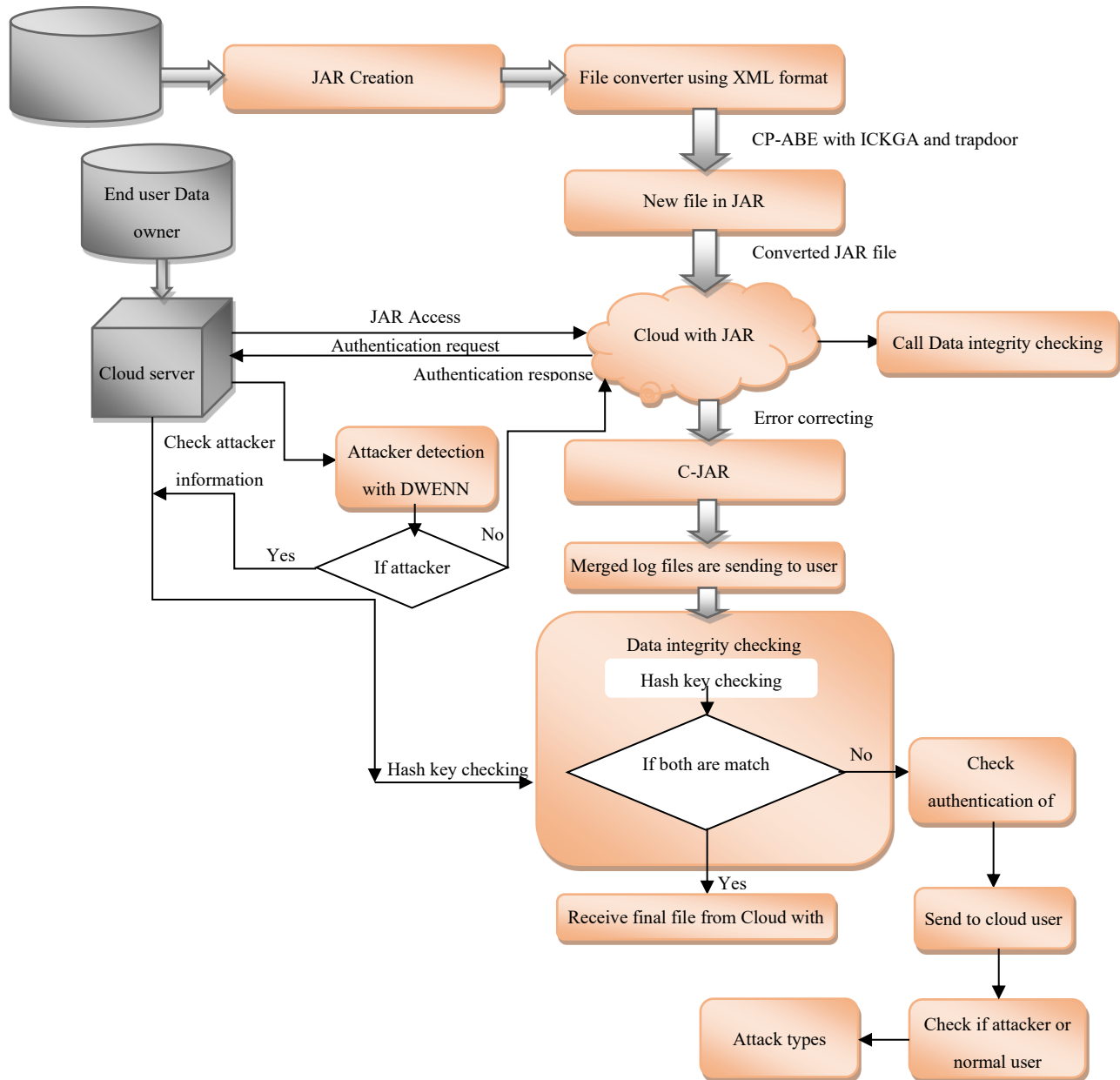


Figure 1: Overview of the CCIAI-ICKGA framework and attack detection

3.1 Components CCIAI-ICKGA Framework

The recommended CCIAI-ICKGA framework is not based on the platform and moreover is hugely decentralized, in the part that there is no requirement for any specific authentication mechanism or storage system. This comprises five vital elements, which include data conversion, logger, key generation, data integrity, log harmonizer, and attack detection.

3.1.1 *Logger*

The particular element, logger, is fundamentally the element that is attached firmly to the data of the customer, with the objective that when the data is accessed, it gets downloaded, and then it is copied every time the data is copied. With a specific instance or a data copy of the users, it seamlessly operates and is considered accountable for recording the access log for a specific case or its duplicate. The logger is sturdily combined with the data of the users, irrespective of whether it is a single data item or whether it is a host of data. The primary functionality of this comprises automatic logging access made to data items, which include the following: encoding the record that maintains a log and deploys the respective content owner's public key, and every now and then forwarding them to the log harmonizer.

3.1.2 *Data conversion*

Data conversion is referred to as the transformation of one data format into another. This is a technical process usually executed by the software, although occasionally we employ hardware or human intervention in this procedure. The motive behind data conversion is to assist interoperability and also in order to maintain all of the data by embedding additional data.

3.1.3 *Choose the input file*

At this juncture, a file is taken into consideration that is illustrative of the format, and it serves as a template. As soon as the adapter has been developed in opposition to it, it states that a similar definition must work in opposition to any file of a similar format. Now and then, so frequently, there is sufficient context for the stylus studio to recognize the fundamental structure in addition to encoding up front, thus, as a result, it offers to "guess" for you, which are ultimately dependent upon what it knows concerning some regular file formats.

3.1.4 *Key Generation*

Key generation is referred to as the process of forming keys in Ciphertext-Policy based Attribute-based Encryption (CP-ABE). So as to encrypt as well as decrypt any kind of data being encrypted/ decrypted, a key is utilized.

3.1.5 *Data Integrity in the User Level*

In the introductory phase, the data is uploaded; this procedure is expected from its beginning condition for 'N' and 'CS', which refer to the amount of server as well as cloud storage fixed server correspondingly. The authentication of the data is executed from the user level, previous to forwarding or uploading data to the desired destination or cloud server. This is equivalent to the value of the data that is similar after it reaches the cloud server or when it's in the cloud level. And, this is validated with the assistance of the data read protocol on the server, in addition to the user level. Consequently, with the intention of keeping data integrity by forwarding data into the cloud server, it confirms its value in user control.

3.1.6 *Data Integrity in Cloud Server Level*

The only contrast is that all the procedures are executed in the cloud server location, employing trapdoor generation. The modifications are as mentioned in the following way. Presume that $CS_u \Rightarrow CS$. It is executed for identical data identification in the midst of user as well as server-level data content. Then, the data is uploaded by CS into the server that is part of cloud computing.

3.1.7 *Log harmonizer*

The log harmonizer is responsible for executing the responsibility for constructing the central entity that authorizes the user to have admission to the log files. The master key is essentially

produced by the log harmonizer because it is the authenticated element. It strictly abides by the decryption key for the ICKGA key pair, owing to its responsibility for the log's decryption. Rather, the decryption is executed for the respective client lest the chosen path prevails amid both respective clients, as well as the log harmonizer is not considered to be trustworthy. At this instant, the harmonizer forwards the key to the client in a manner that ensures a protected key exchange. Conversely, execution of the decryption process is done for the respective client side when the chosen path is in the midst of the log harmonizer, as well as the client is not considered trustworthy. Now, the key to the user is forwarded by means of the harmonizer in a safe key exchange that is safe. It assists two auditing techniques, namely, push and pull, that are explained in the recent work [21].

3.1.8 Automated Logging Mechanism

The Logger component is fundamentally a nested Java JAR file that retains the data items of the users in conjunction with the respective log files. The suggested JAR file comprises one external JAR with one or multiple internal JARs.

3.1.9 Logger structure

The authentications of the entities would be handled by the outer JAR, which is taken to be its key accountability that ought to have access to the information kept in the JAR file. Every internal JAR consists of the enciphered data along with the class files to aid in the recovery of log files and also to show the encompassed data in an appropriate format along with a log file for holding the entire enciphered item with two options, for example, pure log and access log [22]. The log records would be generated with the help of the logger component. Logging happens at the time of any form of access that is attempted at the data present in the JAR, and fresh log entries are integrated consecutively, in the sequence of generation $LR=\{r_1, \dots, r_k\}$. In the latest work, a supplementary detail of the logger component is discussed [21].

3.1.10 JARs Availability

In order to protect in contradiction of attacks caused on offline JARs, the CCAI consists of a log harmonizer that comprises two foremost tasks: one is to manage the replicas of JARs, and the other is to reconstruct the logs that are corrupted. Every log harmonizer is accountable for copies of logger components that consist of a similar collection of data items. The harmonizer is carried out in the form of a JAR file. The harmonizer maintains the error correction info transmitted from its logger components, and the ICKGA decryption key of the users, for the purpose of having the log records decrypted, and also for dealing with any duplicate records that were observed from the duplicates of the JARs connected with the customer's data. As a result, the data of the users is robustly merged along with constituents of the logger present in any data JAR file, and the respective logger gets replicated in coordination with the user's data.

3.2 CP-ABE with ICKGA

Consider $\{cu_1, cu_2, \dots, cu_n\}$ is a collection of cloud users as well as their log files $LF = \{lf_1, \dots, lf_n\}$. A set $A2\{cu_1, cu_2, \dots, cu_n\}$ is monotone when $\forall B, C$: if $B \in A$ and $B \subseteq C$ after that $C \in A$. The sets present in 'A' are referred to as the authorized sets, and the sets that are not present in A are referred to as the unauthorized sets. A ciphertext-policy attribute-dependent encryption [22] comprises four basic algorithms, that is, Setup, Encrypt, KeyGen, and Decrypt [21]. In the case of CP-ABE, the ciphertexts are detected with access structures in addition to the private keys that contain attributes. As stated by the security specification, the opponent would desire to be asked over an encryption to an access structure A^* and request for any private key prK so that S does not meet S^* . The Imperialist Competitive Algorithm (ICA) begins with a preliminary population of cloud users with private keys. The optimal private keys are selected and assumed to be countries that are imperialist,

and the remaining form imperialist colonies collectively. Colonies that constitute early private keys of cloud users are split amongst the imperialist countries depending on their level of privacy. After splitting all imperialist colonies, these move closer to their respective imperialist countries. The ultimate privacy of all the conglomerates is dependent on the privacy of the individual private keys of the imperialist countries and the privacy of colonies being considered herein [23]. Once imperialist competition commences among conglomerates, any empire that does not have the ability to stay in this competition and cannot enhance its privacy of each cloud user gets ruled out from the imperialist contest. This leads to the alleviating of privacy of cloud user territories and results in a decrease in lesser privacy of empires that are considered weak. The empire that is found to be the weakest among empires is most likely to lose its cloud user data to a certain extent and, finally, will fall one after another. All of the countries will ultimately become a state, where there exists just one empire in the whole world, and the remaining countries become colonies of that empire [24-25]. Figure 2 depicts the ICA flowchart.

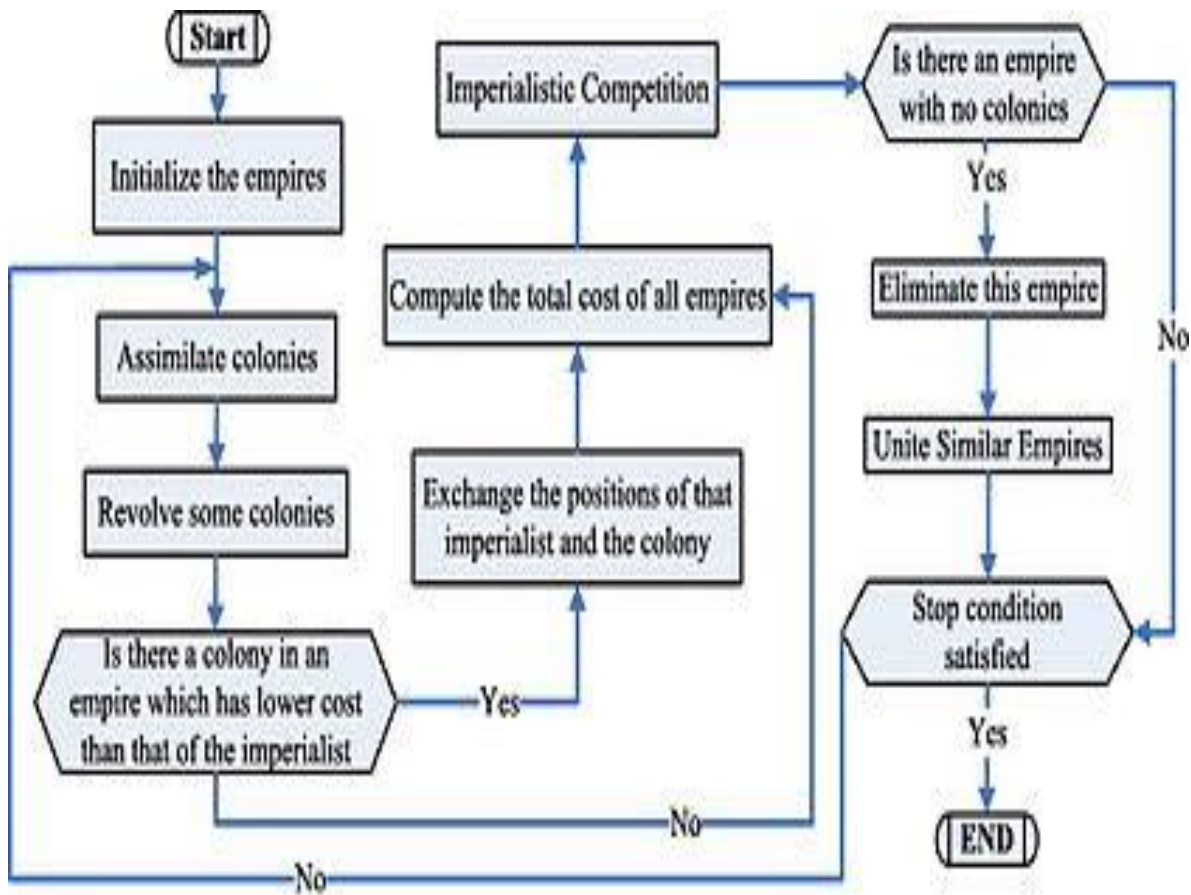


Figure 2: Flowchart of Imperialist Competitive Algorithm (ICA)

The algorithm executes with the following steps (evolution, Competition) till a termination condition is achieved. The derivations pertaining to the empires being created (Initialization) are stated as follows [25]:

$$\text{Country} = [pr_1, \dots, pr_{N_{var}}] \quad (1)$$

$$cost = f(country) = f(pr_1, \dots pr_n) \quad (2)$$

$$N_{col} = N_{pop} - N_{imp} \quad (3)$$

$$C_n = \max_i \{c_i\} - c_n \quad (4)$$

$$p_n = \frac{C_n}{\sum_{i=1}^{N_{imp}} C_i} \quad (5)$$

$$NC_n = Round \{p_n, N_{col}\} \quad (6)$$

The Equation marked as (1) denotes a country, N_{var} represents the magnitude of the cloud users' private keys while p_i represents i-th user's privacy value with their private key, Equation shown as (2) indicates the country's cost, and Equation marked (3) is essential to get the magnitude of the private keys of the population, aids in obtaining the imperialist's normalized cost, in which C_n refers to the nth cost of the imperialist, and in (4) the C_n refers to the privacy value pertaining to the nth imperialist that has been normalized, Equation numbered (5) receives each private keys'(imperialists) power where p_n refers to the nth imperialist's power also Equation (6) indicates the magnitude of the colonies that will be able to contain the imperialist. The set of equations for the transition of the colonies in the direction of their imperialist empire (assimilation) is given by [26]:

$$x \sim U(0, \beta d) \quad (7)$$

$$\theta \sim U(-\gamma, \gamma) \quad (8)$$

Equation (7) stands for a colony, which moves through a distance x in the direction of its imperialist, where β denotes a number between 1 and 2 and 'd' refers to the distance between privacy level between cloud users prevalent in the midst of the colony and its corresponding imperialist, the Equation (8) assists in searching for varied kinds of positions surrounding the imperialist, wherein γ is the factor taken into consideration for estimation of a big value in a global search of the privacy values of all the cloud users and a small value influences the local search of privacy values of all the current cloud users. The Equation that represents the full privacy value of a country is given by [27]:

$$TC_n = Cost(imp) + \xi mean\{cost(col)\} \quad (9)$$

This equation stands for the nth empire's total privacy value (cost), and ξ refers to a value between the numbers 0 and 1. In this regard, the smaller the value of ξ , the higher is the influence of the privacy value of the imperialist to establish the entire privacy value of the empire, and accordingly, the bigger the value of ξ , the greater is the influence over the average privacy value of the referred colonies pertaining to the evaluation of the empire's total privacy value. The set of equations derived for the imperialist contest is as follows [28]:

$$NTC_n = \max_i \{TC_i\} - TC_n \quad (10)$$

$$p_{p_n} = \frac{NTC_n}{\sum_{i=1}^{N_{imp}} NTC_i} \text{ where } \sum_{i=1}^{N_{imp}} p_{p_i} = 1 \quad (11)$$

Equation (10) begins with the contest amongst the imperialists, wherein the total privacy value (cost), which is normalized, is estimated, and Equation (11) defines the probability involved in possessing a colony [27-28]. Let $p^{(0)} = sd^{(0)}$, wherein s is referred to as an invariant used for ease of computation. $p(t)$ includes the scores for which the walker stays at each node at time t , and this consists of the iterative formula

$$P^{t+1} = (1 - r)D^T P^{(t)} + rP^{(0)} \quad (12)$$

Iterating the iteration involves many steps, even now, $p(t)$ is constant, and obtain the steady state P^t . The random walk model is insensitive to the parameters. As a result, set default values for the parameters to be $r=0$, as well as $s=1000$.

3.3 Trapdoor Generation

A family of trapdoor hash [29-30] includes a pair (I, H) , wherein I is referred to as a probabilistic polynomial-time key generation algorithm and H is called a family comprising randomized hash functions. I make a pair (HK, TK) , in which HK is identified as a hash key and TK indicate its related trapdoor key. A trapdoor hash function in H is recognized as a hash function possessing a trapdoor secret. It is represented by $hHK(m, s)$, in which HK is known as a public hash key for user log files, TK is called a private trapdoor key for user log files, m is called a message, and s is known as an auxiliary random number. A trapdoor hash family (I, H) accomplishes the subsequent three characteristics, namely:

(i) Efficiency: Specified HK and any tuple (m, s) , the hash value $hHK(m, s)$ is calculated proficiently (that is to represent, in polynomial time).

(ii) Trapdoor Collision: On the basis of a trapdoor one-way characteristic, we can generate collisions in polynomial time using the entity with the information about the trapdoor secret.

(iii) Collision Resistance: This characteristic is considered from the view of an extra cloud user (for example, an entity lacking the information about the trapdoor key). With possessing no knowledge about the trapdoor secret, it is computation-wise impractical for another cloud user to produce any two random points $(m1, s1)$ and $(m2, s2)$ with the objective that these two points should intersect, which is to represent $hHK(m1, s1) = hHK(m2, s2)$, where $m1 \neq m2$.

In this novel technique, the trapdoor hash function in H relies on the Discrete Logarithm (DL) assumption, which has been defined. Choose at random a big prime q . Choose at random a safe prime p (which is to represent a prime p so that $q = (p - 1)/2$ is prime) and an element g of order q . Select a random element x and also calculate $y = gx \pmod{p}$. The public hash key is represented as $HK = (p, g, y)$, and the private trapdoor key is computed $TK = x$. The trapdoor hash function $hHK(m, s)$ is expressed as below:

$$h_{HK}(m, s) \stackrel{\text{def}}{=} g^m y^s \pmod{p} \quad (13)$$

To exhibit that the $hHK(m, s)$ is referred to as a trapdoor hash function under the DL supposition, demonstrate that the three key functions of a trapdoor hash function are met is necessary, in order for it to have precise efficacy, trapdoor collision, and also collision resistance.

3.4 Attack Detection

In the current work, the following four types of attacks are detected via the application of the Neural Network (NN) classifier.

3.4.1 Disassembling Attack

The section shows suggested system is secured by analyzing the nature and possibilities of attacks that are aimed at disassembling the logger JAR file and thereafter attempting to extort user-

related information from the same or attempting to disrupt or damage log records. Let us consider the disassembling of JAR files from an attacker's perspective, and this is advantageous as it is relatively easy, and on the whole, it poses a significant impact on threats to the CCIAI-ICKGA framework overall. As soon as the disassembling of the JAR files takes place, it enables the attacker to take hold of the public CP-ABE key utilized for the purpose of the log files encryption, the encrypted log file itself, and the *.class files. Hence, to read the log records, the attacker needs to be dependent on knowing about the private key or destroying the encryption. In order to compromise on the log files' confidentiality, the attacker might attempt to recognize which encrypted log records pertain to his actions by the process of mounting a selected plaintext attack for the purpose of obtaining a few pairs of encrypted log records as well as simple texts. The CCIAI framework in the random oracle model has both types of security, which include chosen ciphertext type and chosen plaintext type. As a result, the attacker involved cannot perform decryption on any of the data or log files available in the JAR file that was disassembled. Although the particular attacker might be a certified user, he will have access only to the original content file, but he will not be capable of decrypting any other data, inclusive of the log files that are observable only by the data owner. Using the JAR files that are disassembled, the involved attackers will not possess the ability to analyze the policies involved in the access control directly, this is because the JAR files do not contain the unique source code. Supposing the attacker desires to deduce the policies involved in access control, the solely possible method is by the process of log file analysis. Nevertheless, this is very difficult to achieve since, as mentioned before, the log records get encrypted, and it is computationally difficult to break the encryption. Also, the attacker will not be able to perform modifications on the extraction of the files pertaining to the log from a JAR that has been disassembled. If an attacker intends to wipe out or corrupt any record, the supplemental checks for integrity checks present in each of the records of the log will not be similar during the verification period, thereby resulting in error revelation. Likewise, the attackers do not possess the ability to put down or include fake records in the daily files pertaining to the log without going unnoticed, this is because it requires signing using a legitimate key, as well as there will be no match for the chain of hashes.

3.4.2 Man-in-the-Middle Attack

This section entails details about the attacker who wishes to intercept messages or disrupt the authentication process to gain leverage or access from a service provider who has an authorized and certified access and responds to messages in the pretense of being the actual or genuine service provider. Two points are inherent with respect to time when any attacker can actually replay relevant intercepted messages. First is when the legitimate service provider completes and ends a session and is totally disconnected from the concerned certificate authority. Second instance is while the legitimate service provider has completed the session and has disconnected, however, the session is not in a state of total completion. In this case attacker has the opportunity to reconnect through a meticulously calculated process of renegotiation. In the first case, the success of the attempted attack is most unlikely to occur as the certificate is usually marked with a time stamp that is most likely to turn obsolete at the reuse attempt. In the second scenario, the attack is likely to fail as, in general, there is an established ban on renegotiation, specifically in the latest OpenSSL version, and additionally, with cryptographic checks being in place, failure is assured.

3.4.3 DDoS attacks

HTTP DDoS attacks are separated into two main categories: high-rate and low-rate attacks. In high-rate HTTP DDoS attacks, frequently known as HTTP flooding attacks, a massive number of HTTP requests flood the target. These attacks are simple flooding attacks that produce as many HTTP request packets by utilizing all the available resources of the Reflectors. Their behavior is extremely

similar to the traditional DoS attacks, which result in a total consumption of the victim's resources and/or exhaustion of the communication channel.

3.4.4 Slow Message Body Attack (Slow POST)

When a legitimate client intends to send data to a web server using the POST method, after the establishment of a TCP connection, it first sends an HTTP message header in which it states the amount of data to be sent. The type of field used to communicate this information to the web server is the Content-Length. Upon the server receiving the message header, it reads the Content-Length value. Subsequently, it keeps the connection alive as well as the resources running until obtaining the total volume of data mentioned in the Content-Length field. In its turn, the client splits the message body into chunks according to its network resources.

3.4.5 Copying Attack

The instinctive attack is when attackers copy complete JAR files. Attackers are most likely to presume that this would provide easy access to data present in the JAR file, and they would not be noticed while intruding and in doing so by the data owner. Such kind of attacks, though, are detected when deploying the auditing mechanism suggested in this study. Let us remind ourselves that each JAR file needs to transmit the log records to the harmonizer. Specifically, in relation to the push mode, the harmonizer sends logs to data owners intermittently. Provided the data owner is unaware of the inherent extra copies of its JAR files, log files from all available copies will be sent and received by the data owner, respectively. Considering attackers shift JAR copies to such places where the harmonizer is unable to connect, JAR copies in that scenario turn out to be unreachable. It happens as every JAR needs the redundancy information to harmonizer to be recorded regularly. Assuming the JAR is unable to contact the harmonizer, access to JAR content gets disabled. Hence logger component is more transparent than other traditional log file encryption, facilitating the data owner to identify if an attacker has made copies of a JAR, and it makes the offline files to become inaccessible.

3.4.6 Dynamically Weighted Ensemble Neural Networks (DWENN)

The methods collectively integrate outputs of different neural networks [31]. Collective output is a weighted average of the outputs of every network, with the ensemble weights decided in the form of a function of the relative error of every network decided in training [31]; the resultant network frequently performs better than the constituent networks. Let us take a population of networks that are trained over a set comprising labeled cloud users' information of a binary attack detection problem into consideration. The renowned and usual method known as the general ensemble method (GEM) may be expressed as follows:

$$f_{GEM} = \sum_{i=1}^n \alpha_i f_i(x) \quad (14)$$

Wherein α_i are selected for the purpose of minimizing Mean Square Error (MSE) instead of attack detection 'f' and sum to 1. Hence error $\epsilon_i(x)$ of a network f_i can be represented as

$$\epsilon_i(x) = f(x) - f_i(x) \quad (15)$$

Define the correlation matrix $C_{ij} = E[\epsilon_i(x)\epsilon_j(x)]$. Then must find weights α_i that minimize

$$MSE[f_{GEM}] = \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j C_{ij} \quad (16)$$

It can be shown [31] that the optimal choice for the

$$\alpha_i = \frac{\sum_{j=1}^n C_{ij}^{-1}}{\sum_{k=1}^n \sum_{j=1}^n C_{kj}^{-1}} \quad (17)$$

This method yields better performance for attack detection of cloud users. The technique is based on making an exact estimate of C that is reliable, and considering C as being a non-singular inversion is easy [30]. Practically, there is an inherent correlation between errors, and hence, C rows are assumed to be in a state of near linear dependency such that inversion of C will result in inevitable round-off errors. Assuming a neural network's output 'y=attacker' and its interpretation may be considered as the cloud user information x is present in a class, then y approaches 1=attacker, it becomes more certain that the instance is present in the class. When 'y' approaches 0=no attack, it becomes more possible that the cloud user information is not present in the class. Quantifying this idea as follows: the certainty(y) of a neural network is defined as

$$c(y) = \begin{cases} y & \text{if } y \geq \frac{1}{2} \\ 1 - y & \text{else} \end{cases} \quad (18)$$

Let us consider that the certainty behaves symmetrically concerning positive and negative decisions; the certainty of an output of 0.1 is similar to that of an output of 0.9, however, the attack detection decision they are certain about is not the same. If, in place of selecting static weights obtained from f_i performance on cloud user information of the input space, lets the weights to adapt so that they are proportional to certainties of network outputs that are taken into consideration, this will help attain an improved performance. Hence, dynamically defining the averaged network may be said as under

$$f_{DAN} = \sum_{i=1}^n w_i f_i(x) \quad (19)$$

The w_i sum to 1, hence f_{DAN} may be said to be a network output weighted average. The primary differentiating factor is the weight vector, which is recalculated every time evaluation of ensemble output is done, so that the best attack detection for the respective cloud user information that is being considered is carried out, in place of statically choosing weights that provide optimal detection for the cross-validation set. The sum contribution of each network may be said to be proportional with respect to its certainty. Selection of a value near 0.5, for cloud user information, can just have some negligible contribution to the respective sum; however, a very certain value of 0.99 (or 0.01) amid several less certain values that dictate the respective sum. The technique resembles deploying an agreement between a set of classifiers to attain a measure of confidence in the attack detection. However, the confidence level (certainty) of every classifier is deployed to execute the final attack detection.

4 Results and Discussion

While executing the examination session, the test condition area must be collated, and subsequent analysis of the Centralized Information Accountability (CIA), CCIAI - Firefly Key Generation Algorithm (FKGA), CCIAI-Bat Key Generation Algorithm (BKGA), and proposed CCIAI-ICKGA system is presented by fixing up a small CC. The Open Java Development Kit (JDK) runtime environment allows the successful implementation of techniques. The degree at a time allows the overhead to occur at three motivates: at the time of authentication, then at the log record, and for the period of the combining of the logs.

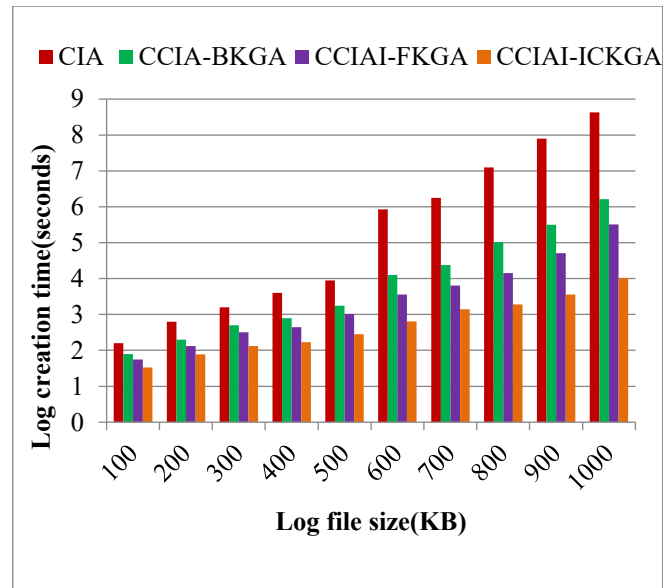


Figure 3: Time to generate Log Files of various Sizes vs. key generation methods

Figure 3 depicts time is likely to increase to create the log file, along with the log file size. Primarily, the CCIAI-ICKGA system suggested, the time to yield a 100 KB file is approximately 1.52 seconds, and a 1000 KB file is around 4.01 seconds.

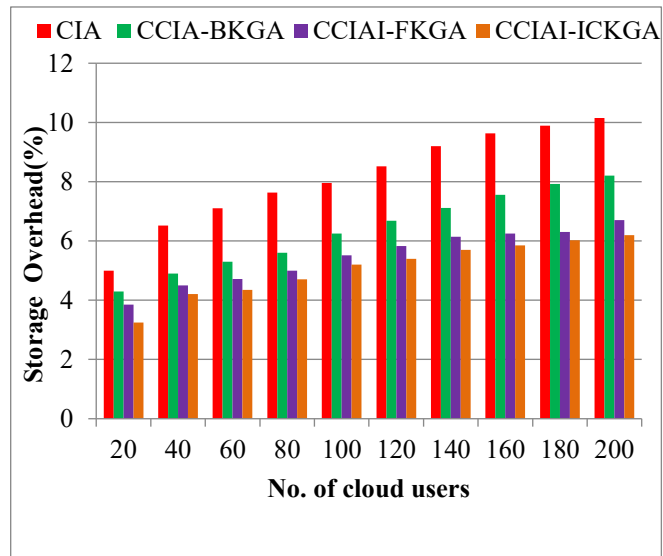


Figure 4: Storage overhead at different cloud users vs. key generation methods

Figure 4 depicts the storage overhead results of four different key generation algorithms. The storage overhead of the proposed CCIAI-ICKGA system is around 6.2% for 200 cloud users, whereas other methods, such as the CIA framework, are around 10.15%, the CCIA-BKGA algorithm is around 8.21% and the CCIAI-FKGA system is around 6.7%. Hence, it has been proved with the suggested approach that minimum storage space is required in comparison with the results from other techniques.

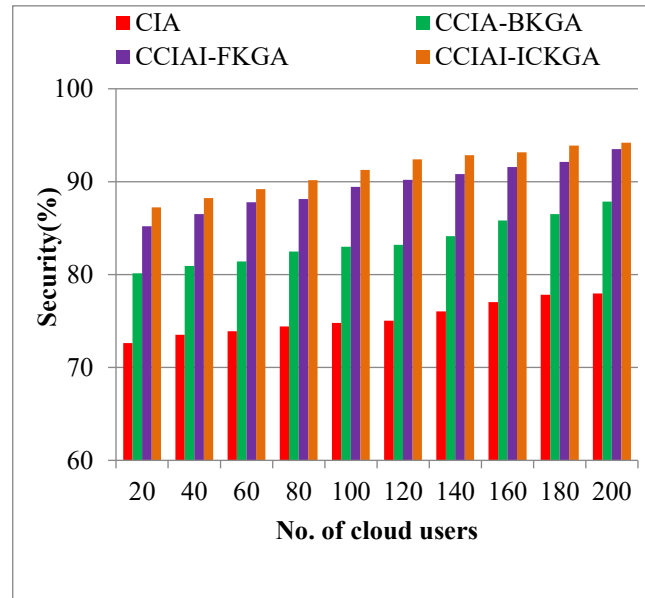


Figure 5: Cloud security at different cloud users vs. key generation methods

Figure 5 depicts the security level of several authentication approaches. The CCIAI-ICKGA system suggested, and its security level is around 6.2% for 200 cloud users, whereas other methods, such as the CIA framework, are around 10.15%, the CCIA-BKGA algorithm is around 8.21% and the CCIAI-FKGA system is around 6.7%.

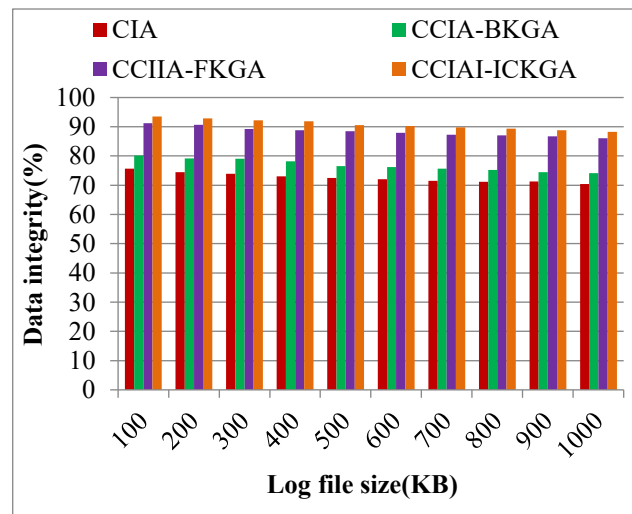


Figure 6: Data integrity at different cloud users vs. key generation methods

Figure 6 shows the results from data integrity performed under different sizes of the log file. CCIAI-ICKGA system data integrity level is around 93.52% for a 100 KB file, whereas other methods, such as the CIA framework, are around 75.63%, CCIA-BKGA algorithm is around 80.15% and CCIAI-FKGA system is around 91.25% for a 100 KB file.

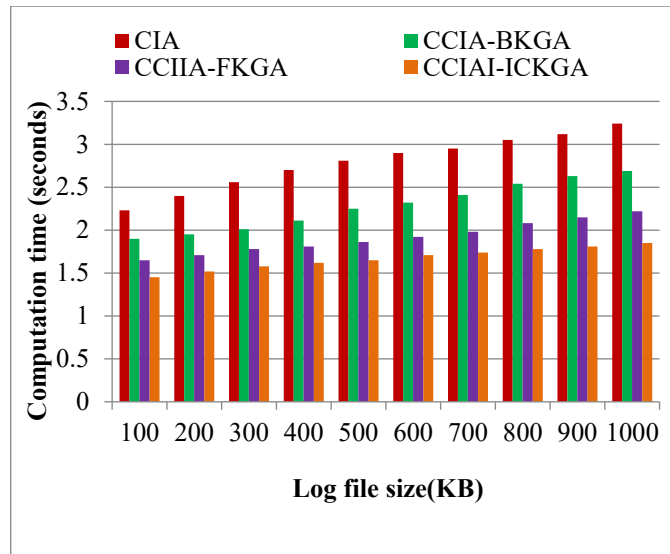


Figure 7: Computation time at different cloud users vs. key generation methods

Figure 7 shows the results drawn from the computation time under different sizes of the log file. CCIAI-ICKGA system computation time that has been suggested is around 1.85 seconds for a 100 KB file, whereas other methods, such as the CIA framework, are around 3.24 seconds, the CCIA-BKGA algorithm is around 2.69 seconds, and the CCIAI-FKGA system is around 2.22 seconds for a 100 KB file.

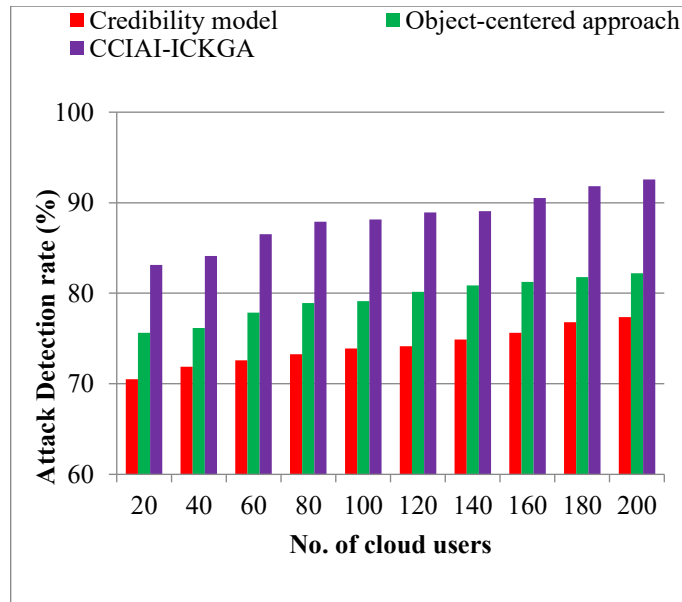


Figure 8: Attack Detection Rate at different cloud users vs. key generation methods

Figure 8 shows attack detection rate results under different numbers of cloud users. The DWENN system suggested and its attack detection rate is around 92.56% for 200 cloud users, whereas other methods, such as the credibility model [16], are around 77.36%, and object object-centered approach is around 82.21% for 200 cloud users.

5 Conclusion

The research work introduced suggests an innovative attack detection and Centralized Cloud Information Accountability Integrity with Imperialist Competitive Key Generation Algorithm (CCIAI-ICKGA), which works automatically when logging or accessing data present in the cloud, and also deploys an effective auditing strategy. The CCIAI-ICKGA approach facilitates data owners to both audit content as well as enforce strong back-end protection as and when required. Additionally, the study entails an analytical analysis of attack detection and the strength of CCIAI-ICKGA architecture when faced with varying non-trivial attacks, which are initiated by meticulously shrewd users or occur because the JRE, also known as Java Running Environment, has been compromised somewhere. Dynamically Weighted Ensemble Neural Networks (DWENN) classifier performs the attack detection; it detects attacks inside JRE automatically. Most attacks are rendered ineffective with the DWENN classifier, as it automatically protects against the most common attacks. Log file encryption inhibits attackers from accessing any unauthorized information. The CCIAI-ICKGA framework that has been suggested inhibits attacks like the ones involving the detection of copies of users' data, which is considered illegal. CCIAI-ICKGA work vastly varies from conventional logging techniques that encrypt data to protect log files. The research and future studies should be geared to encompass several policies pertinent to data and information security, as indexing policies that are applied in large text files, controlled use of execution commands, and a host of controls that determine accountability and governance.

Acknowledgement: Not applicable.

Funding Statement: Not applicable.

Author Contributions: The authors confirm contribution to the paper as follows: Conceptualization, methodology and writing—original draft preparation: Niranchana Radhakrishnan; supervision and writing: C. Viji; review and editing: Balusamy Nachiappan. All authors reviewed the results and approved the final version of the manuscript.

Conflicts of Interest: The authors declare no conflicts of interest to report regarding the present study.

References

1. Jaeger PT, Lin J, Grimes JM. Cloud computing and information policy: Computing in a policy cloud?. *J Inf Technol Polit.* 2008;5(3):269-83.
2. Pearson S, Charlesworth A. Accountability as a way forward for privacy protection in the cloud. In *IEEE international conference on cloud computing*; 2009 Dec 1, Berlin, Heidelberg: Springer Berlin Heidelberg; 2009. p. 131-144.
3. Wei L, Zhu H, Cao Z, Dong X, Jia W, Chen Y, Vasilakos AV. Security and privacy for storage and computation in cloud computing. *Inf Sci.* 2014;258:371-86.
4. Reed A, Rezek C, Simmonds P. 2011. Security guidance for critical areas of focus in cloud computing v3.0. CSA. 2011. p. 14-44.
5. Chen D, Li X, Wang L, Khan SU, Wang J, Zeng K, Cai C. Fast and scalable multi-way analysis of massive neural data. *IEEE Trans Comput.* 2014;64(3):707-19.
6. Khan AN, Kiah MM, Madani SA, Ali M, Khan AU, Shamshirband S. Incremental proxy re-encryption scheme for mobile cloud computing environment. *J Supercomput.* 2014;68(2):624-51.
7. Roughgarden T. Algorithmic game theory. *Commun ACM.* 2010;53(7):78-86.
8. Ren K, Wang C, Wang Q. Security challenges for the public cloud. *IEEE Internet Comput.* 2012;16(1):69-73.
9. Gonzalez N, Miers C, Redigolo F, Simplicio M, Carvalho T, Näslund M, Pourzandi M. A quantitative analysis of current security concerns and solutions for cloud computing. *J CLOUD COMPUT-ADV S.* 2012;1(1):11.

10. Moore D, Shannon C, Brown DJ, Voelker GM, Savage S. Inferring internet denial-of-service activity. *ACM Trans Comput Syst.* 2006;24(2):115-39.
11. He Z, Zhang T, Lee RB. Machine learning based DDoS attack detection from source side in cloud. In 2017 IEEE 4th International Conference on Cyber Security and Cloud Computing (CSCloud) 2017 Jun 26, New York, NY, USA; 2017, p. 114-120.
12. Sreeram I, Vuppala VP. HTTP flood attack detection in application layer using machine learning metrics and bio inspired bat algorithm. *Appl Comput Informatics.* 2019;15(1):59-66.
13. Sofi I, Mahajan A, Mansotra V. Machine learning techniques used for the detection and analysis of modern types of ddos attacks. *Int Res J Eng Technol.* 2017;4(6):1085-92.
14. Choi J, Choi C, Ko B, Kim P. A method of DDoS attack detection using HTTP packet pattern and rule engine in cloud computing environment. *Soft Comput.* 2014;18(9):1697-703.
15. Aiello M, Cambiaso E, Mongelli M, Papaleo G. An on-line intrusion detection approach to identify low-rate DoS attacks. In 2014 International Carnahan Conference on Security Technology (ICCST); 2014 Oct 13, Rome, Italy; 2014. p.1-6.
16. Noor TH, Sheng QZ, Alfazi A. Reputation attacks detection for effective trust assessment among cloud services. In 2013 12th IEEE international conference on trust, security and privacy in computing and communications; 2013 Jul 16, Melbourne, VIC, Australia; 2013. p. 469-476.
17. Sundareswaran S, Squicciarini A, Lin D. Ensuring distributed accountability for data sharing in the cloud. *IEEE Trans Dependable Secure Comput.* 2012;9(4):556-68.
18. Garkoti G, Peddoju SK, Balasubramanian R. Detection of insider attacks in cloud based e-healthcare environment. In 2014 International Conference on Information Technology; 2014 Dec 22, Bhubaneswar, India; 2014. p. 195-200.
19. Ko RK, Jagadpramana P, Mowbray M, Pearson S, Kirchberg M, Liang Q, Lee BS. TrustCloud: A framework for accountability and trust in cloud computing. In 2011 IEEE World Congress on Services; 2011 Jul 4, Washington, DC, USA; 2011. p. 584-588.
20. Hwang K, Li D. Trusted cloud computing with secure resources and data coloring. *IEEE Internet Comput.* 2010;14(5):14-22.
21. Punitha AA, Indumathi G. Centralized cloud information accountability with bat key generation algorithm (CCIA-BKGA) framework in cloud computing environment. *Cluster Comput.* 2019;22(2):3153-64.
22. Waters B. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In International workshop on public key cryptography; 2011 Mar 6, Berlin, Heidelberg; 2011. p. 53-70.
23. Khabbazi A, Atashpaz-Gargari E, Lucas C. Imperialist competitive algorithm for minimum bit error rate beamforming. *Int J Bio-Inspired Comput.* 2009;1(1-2):125-33.
24. Enayatifar R, Sadaei HJ, Abdullah AH, Gani A. Imperialist competitive algorithm combined with refined high-order weighted fuzzy time series (RHWFTS-ICA) for short term load forecasting. *Energy Convers. Manag.* 2013;76:1104-16.
25. Shamshirband S, Amini A, Anuar NB, Kiah ML, Teh YW, Furnell S. D-FICCA: A density-based fuzzy imperialist competitive clustering algorithm for intrusion detection in wireless sensor networks. *Measurement.* 2014;55:212-26.
26. Atashpaz-Gargari E, Lucas C. Imperialist competitive algorithm: an algorithm for optimization inspired by imperialistic competition. In 2007 IEEE congress on evolutionary computation; 2007 Sep 25, Singapore; 2007. p. 4661-4667.
27. Nourmohammadi A, Zandieh M, Tavakkoli-Moghaddam R. An imperialist competitive algorithm for multi-objective U-type assembly line design. *J Comput Sci.* 2013;4(5):393-400.
28. Hosseini S, Al Khaled A. A survey on the imperialist competitive algorithm metaheuristic: implementation in engineering domain and directions for future research. *Appl Soft Comput.* 2014;24:1078-94.
29. Yang FY. Improvement on a Trapdoor Hash Function. *Int J Netw Secur.* 2009;9(1):17-21.
30. Harn L, Hsin WJ, Lin C. Efficient on-line/off-line signature schemes based on multiple-collision trapdoor hash families. *Comput J.* 2010;53(9):1478-84.
31. Das R, Turkoglu I, Sengur A. Effective diagnosis of heart disease through neural networks ensembles. *Expert Syst Appl.* 2009;36(4):7675-80.
32. Maqsood I, Khan MR, Abraham A. An ensemble of neural networks for weather forecasting. *Neural Comput Appl.* 2004;13(2):112-22.